

MINESHIELD

OFFLINE CAPABILITIES DOCUMENTATION

Computer Programming I - Group 16

May 29, 2026

1 Introduction to Offline-First Architecture

MineShield is designed for hazardous industrial environments, particularly underground mines and deep construction sites where continuous network connectivity is rarely guaranteed. To maintain uninterrupted safety monitoring, the application is built with an “offline-first” architecture. This document outlines how the application handles network drops and serves as a guide for developers implementing new features that require offline support.

2 Application Behavior When Offline

When a device loses its network connection, MineShield ensures that worker safety operations continue without interruption:

- **Uninterrupted Workflow:** The user is not blocked by network error screens. They can continue to navigate cached data and perform necessary safety actions.
- **Action Queuing:** Instead of failing with network errors, data-modifying actions (such as logging a hazard report or updating worker status) are intercepted and securely saved to a local offline queue via AsyncStorage.
- **User Interface Feedback:** The UI displays a persistent “Offline Mode” indicator, reassuring users that their data is stored locally and will be synchronized later.

3 Automated Synchronization (useOfflineSync)

The background synchronization engine is driven by the useOfflineSync hook, which acts as the core recovery mechanism for the application.

- **Network State Listening:** The hook continuously monitors the device’s network state using the React Native NetInfo module.
- **Queue Processing:** The moment a stable internet connection is detected, useOfflineSync automatically triggers a background worker.
- **Sequential Execution:** The worker processes the offline queue chronologically (First In, First Out) to ensure data consistency with the backend server. Once an action successfully syncs with the cloud, it is permanently purged from the local device queue.

4 Developer Guide: Implementing Offline Features

When building new features that require backend communication, developers must ensure graceful degradation when offline. Instead of making direct API calls that will fail silently, developers must use the saveToOfflineQueue utility function.

Below is the standard implementation pattern for offline-safe data submission:

```
1 import { saveToOfflineQueue } from '@utils/offlineSync';
2 import { useNetworkState } from '@hooks/useNetworkState';
3 import api from '@api/client';
4
```

```
5 const submitHazardReport = async (reportData) => {
6   const { isOnline } = useNetworkState();
7
8   if (isOnline) {
9     // Standard direct API call if connected
10    try {
11      await api.post('/hazards', reportData);
12      console.log('Hazard reported successfully');
13    } catch (error) {
14      console.error('API Error', error);
15    }
16  } else {
17    // 1. Define a consistent action identifier
18    const actionType = 'SUBMIT_HAZARD';
19
20    // 2. Save the complete payload to the offline queue
21    // Note: Ensure the payload contains everything the server needs!
22    await saveToOfflineQueue(actionType, reportData);
23
24    // 3. Provide user feedback
25    alert('You are offline. Report saved locally and will sync later.')
26  }
27 };
```

Listing 1: Example implementation of saveToOfflineQueue for new features

Best Practices for Queue Management:

- **Action Types:** Always use a distinct, uppercase string for your action type (e.g., SUBMIT_HAZARD). This string must be mapped to the corresponding API route inside the useOfflineSync processor.
- **Payload Completeness:** The payload passed to the queue must contain all necessary identifiers (such as UUIDs, timestamps, and worker IDs) so the server can accurately process it once it arrives, even if hours have passed.